Guía de estudio – Parcial 2 Ingeniería y Calidad de Software (UTN FRC)

Unidad 2 – Filosofía Lean

Lean busca maximizar el valor para el cliente con el mínimo consumo de recursos: todo lo que no crea valor es desperdicio. Nace en la manufactura/industria automotriz (Toyota, década del 40), no en el software; los Poppendieck lo adaptaron al desarrollo de software (libro: Lean software development). Es menos prescriptivo que ágil (ágil tiene pocas reglas, Lean tiene menos). Su lema de partida: *tomá el proceso que tenés y aplicá los principios para mejorarlo*, a diferencia de ágil, que es más prescriptivo. Usa el método Just in Time: producir lo necesario, en la cantidad necesaria y cuando se necesita.

Muchos principios ágiles heredan de Lean. Ambos están orientados al cliente y al valor, buscan equipos motivados, flexibilidad ante el cambio y mejora continua. No hay contraposición, nada que vaya en contra entre ellos, por lo que en la actualidad se habla de enfoques lean-ágil.

Los 7 principios Lean

#	Principio	Idea clave	Relación con ágil
1	Eliminar el desperdicio En el que más se hace énfasis	Todo lo que no agrega valor al negocio (trabajo innecesario, retrabajo, funcionalidades sin uso). Tiempo en lo que no agrega valor debe tender a 0	Software funcionando y simplicidad
2	Amplificar el aprendizaje	Cultura de mejora continua. Pasar conocimiento implícito a explícito y compartirlo en toda la organización. Se relaciona con empirismo	Autogestión de equipos; aceptar cambios de requisitos; técnicos y no técnicos trabajando juntos
3	Embeber la integridad conceptual	Calidad desde el inicio, no inspeccionar después de los hechos. Todas las partes del producto encajan con coherencia y consistencia (arquitectura, requerimientos no funcionales). Integridad percibida (balance de función, uso, confiabilidad y economía) e integridad conceptual (componentes que trabajan coherentes en conjunto). Incluye revisiones entre pares e inspecciones	Calidad no negociable; excelencia técnica
4	Diferir compromisos hasta el último momento responsable	Tomar la decisión recién cuando se tiene la mayor información posible, pero antes de que sea tarde. Se ve en el Product Backlog: las US de abajo no se detallan	Maximizar el trabajo no hecho; aceptar cambios tardíos

#	Principio	Idea clave	Relación con ágil
5	Dar poder al equipo	Equipos autogestionados y multifuncionales. Delegar decisiones al nivel más bajo posible, entrenar líderes, respetar a la gente. Quien hace el trabajo decide cómo	Principio 11: equipos autoorganizados
6	Ver el todo	Visión holística: el producto, el valor detrás y el servicio que brinda. Los objetivos de cada área alineados con los globales. Incluye diseño arquitectónico, un "hueco" de Scrum	–
7	Entregar lo antes posible	Acortar ciclos de desarrollo, incrementos pequeños de valor, salir pronto al mercado con un producto mínimo valioso. Retroalimentación rápida	Entregas tempranas y frecuentes

Los 7 desperdicios (versión software)

Entre corchetes, su equivalente en manufactura.

Desperdicio en software	Qué es
Características extra [producción en exceso]	Funcionalidades que no se usan (80% del valor está en el 20% de las funcionalidades, Pareto). Incluye la especificación anticipada (*up front specification*): cuando se implementa, la información ya está obsoleta o es errónea
Trabajo a medias [stock]	US, bugs o casos de uso sin terminar. Por eso en ágil la gestión es binaria: está hecho o no
Procesos extra	Pasos del proceso que no aportan valor
Búsqueda de información / movimiento	Perder tiempo buscando información. Se combate con gestión de configuración (trazabilidad, acceso fácil)
Defectos	Errores que se trasladan a una etapa posterior y generan retrabajo. Antídoto: aseguramiento de calidad
Esperas [transporte/logística]	Tiempo que una persona espera para poder trabajar (depende de otro, necesita aprobación). Por eso importan los equipos multidisciplinarios
Cambio de tareas (multitasking)	Se pierde tiempo de preparación en cada tarea. Lema: "dejá de empezar y empezá a terminar". Kanban lo ataca limitando el WIP

[Completar] ejemplos propios de cada desperdicio si la clase grabada los desarrolla.

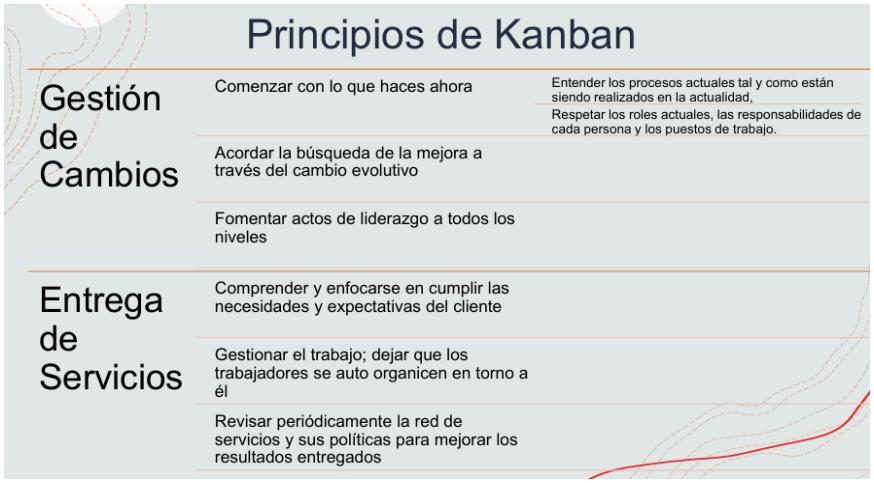
Unidad 2 – KANBAN

Framework referente de la filosofía lean en software. Kanban es un enfoque para gestionar formas de trabajo (procesos) con el fin de lograr mejora continua. NO es un proceso de desarrollo ni una metodología de administración de proyectos: sirve para introducir cambios graduales en un proceso o metodología que la organización ya tiene/ implementar mejoras continuas en las organizaciones. Su principio de partida es *empezá por donde estás*, lo que reduce la resistencia al cambio. Es el referente principal de Lean en software.

Origen: Toyota (fines de los 40) estudió el almacenamiento y el tiempo de stockeo en supermercados, de donde sale Just in Time. *Kanban* significa "tarjeta de señal". A**plica administración de colas** (ejemplo de Starbucks: si hay clientes esperando, el cajero deja de cobrar y ayuda al barista; "dejá de empezar y terminá primero"). Funciona como sistema **pull** (el trabajo se arrastra cuando hay capacidad), no **push**. La idea es brindar información que les hace falta a quienes tienen que participar en el proceso de construcción de algo queremos lograr la visualización del trabajo, del flujo, saber en cada momento del proceso donde está la pieza que debemos entregarle al cliente.

Trabaja sobre una pieza de trabajo individual (US, requerimiento, ticket, defecto, cambio), no sobre iteraciones ni proyectos. Para lograr la visualización utiliza como herramienta un tablero





Prácticas de Kanban

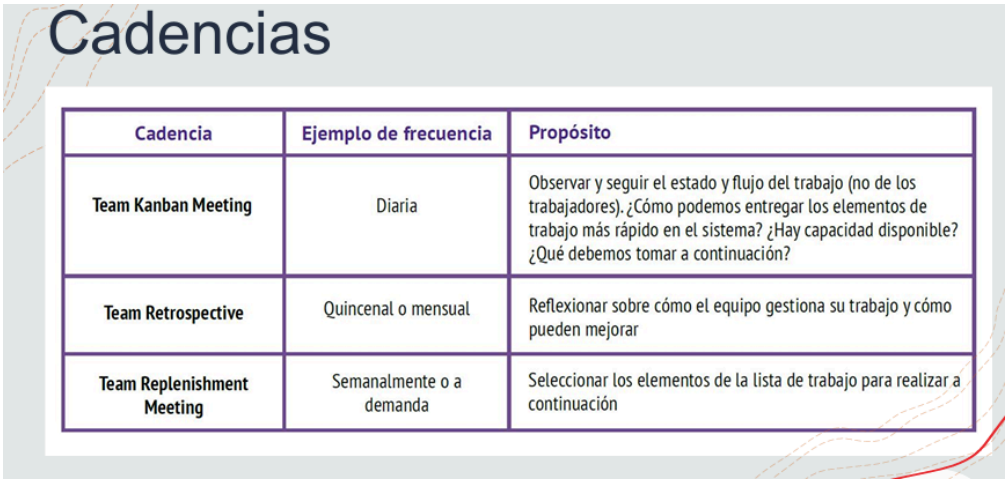
1. **Visualizar el trabajo/flujo**: tablero con columnas que representan las etapas, de izquierda a derecha. Las tarjetas muestran en qué etapa está cada pieza. Da transparencia. Hay columnas de producción (se está trabajando) y de acumulación (listas para pasar a la siguiente etapa).
2. **Limitar el WIP (work in progress)**:Forma de controlar el flujo (en scrum en el timebox) límite explícito de piezas por columna para evitar cuellos de botella y sobrecarga. Es lo que crea el sistema de asignación **pull** o de arrastre. Cual es el número máximo de tareas que pueden recibir en un determinado periodo de tiempo en esa columna
3. **Gestionar el flujo**: buscar flujo continuo e ininterrumpido; identificar cuellos de botella, recursos ociosos, tiempos de espera.

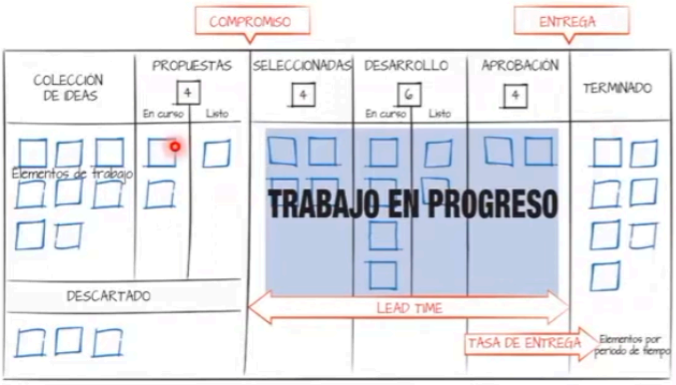
4. **Hacer explícitas las políticas**: pocas, simples, visibles, siempre aplicadas y fáciles de modificar (criterios para pasar de columna, DoD, prioridades, WIP).
5. **Mejorar de forma colaborativa y evolutiva**: mejora continua gradual, sin cambios revolucionarios.
6. **Respetar roles, responsabilidades y habilidades existentes**: no impone roles; el equipo se organiza según sus fortalezas.

El resumen final menciona además los *circuitos de retroalimentación* (reuniones con frecuencia definida) como una práctica; Covaro los trata dentro de la cadencia de actividades clave.

Cómo aplicar Kanban (pasos)

1. Entender el proceso actual y graficarlo en un tablero (empezar con lo que se tiene).
2. Dividir el trabajo en piezas (unidades de trabajo): US, requerimientos, defectos, incidentes, solicitudes de cambio, mantenimiento.
3. Tipificar las piezas (colores por tipo de trabajo) y asignar capacidad según la demanda (ej.: 60% casos de uso, 30% mantenimiento, 10% defectos).
4. Visualizar el flujo con columnas. Las personas se autoasignan las piezas (pull).
5. Limitar el WIP por columna según la capacidad del equipo. Si en una columna se llega al límite y el trabajo no fluye, es un cuello de botella: se mueven recursos de otra área.
6. Definir políticas de calidad por columna (criterios para pasar a la siguiente) y por clase de servicio.
7. Definir clases de servicio. Diferentes trabajos tienen diferentes politicas - definicion de hecho ("done") para cada estado.
8. Monitorear el flujo, capacitar al equipo y establecer una **cadencia** (releases, planificaciones, revisiones). La define el equipo, no hay un lineamiento explícito.





Hay dos tipos de columnas:

De acumulacion: Las que dicen listo o terminado

De trabajo: En curso, haciendo, etc.



Definimos que su proceso se mapeaba al tablero de esta forma:

Cola de Producto	Análisis		Desarrollo		Listo para Build	En Testing		En
	En progreso	Hecho	En progreso	Hecho		En Progreso	Listo para Despliegue	

Se puede tener múltiples carriles para distintas piezas de trabajo, proyectos, etc.

Clases de servicio

El tablero no es una cola FIFO: priorizan según urgencia y fechas.

Las clases de servicio definen cómo se tratan las piezas. Permiten hacer explícitas las políticas de calidad, es decir que comportamiento van a tener en el tablero las unidades de trabajo que nosotros tenemos en KANBAN. Ejemplos:

Clase	Cómo se trata
Expreso - bala de plata (alta prioridad)	Color de tarjeta blanco. Se resuelve lo antes posible. WIP = 1 (Solo puede haber una pieza de este tipo por vez en el tablero). El resto se pone en espera. Puede exceder el límite de WIP para procesar este trabajo. La capacidad no se reserva. De ser necesario se hace una entrega especial, para ponerla en producción
Fecha fija	Color de tarjeta rosa. Debe adherirse al WIP definido. Fecha de entrega en la parte superior. Permanecen la cola hasta que sea conveniente que ingresen. Si se retrasa y la fecha de entrega está en riesgo puede promoverse a la clase de servicio “expreso”. Son entregados en entregas programadas cuidando la fecha de entrega
Estándar	Color de tarjeta: Amarillo. Deben adherirse al WIP definido. Son priorizados y puestos en la cola con un mecanismo definido basado en valor de negocio. Usan la técnica FIFO, si no hay Expresos o con Fecha Fija. Pueden analizarse por tamaño, en orden de magnitud. Son entregados en entregas programadas
Intangible	Color de tarjeta: verde. Deben adherirse al WIP definido. Son priorizados y puestos en la cola con un mecanismo definido basado en valor de negocio. Usan la técnica FIFO, si no hay Expresos o con Fecha Fija. Pueden analizarse por tamaño, en orden de magnitud. Son entregados en entregas programadas Aparece con signo de pregunta en las notas de Covaro. *[Completar]* con la clase grabada

Métricas de Kanban

Como en scrum mide producto, kanban mide proceso. Mide cuánto tiempo tardamos en entregar cada pieza que pasa por el tablero.

Lead time, vista del cliente.

Es la metrica que registra el tiempo que sucede entre el momento en el cual se esta pidiendo un item de trabajo y su momento de entrega (el final del proceso). Se suele medir en dias de trabajo

Ritmo de entrega.

Al no ponerle límite de work in progress al backlog, no se limita la expectativa del cliente y debido a que se pueden acumular tareas en el backlog con mayor prioridad el lead time va a ser mayor, algunos equipos de igual forma no le ponen límite para evitar perdida de ideas…U colocan una columna de colección de ideas sin WIP y el tiempo se comienza a contar una vez que se define el compromiso

Cycle Time = Vista interna, para el equipo.

Es la métrica que registra el tiempo que sucede entre el inicio y el final del proceso, para un ítem de trabajo dado. Se suele medir en días de trabajo o esfuerzo.

Medicion mas mecánica de la capacidad del proceso

Ritmo de terminación

Touch Time (Tiempo de tocado)

Tiempo en el cual un item de trabajo fue realmente trabajado (o tocado por el equipo).

Cuántos días hábiles pasó este ítem en columnas de "trabajo en curso", en oposición con columnas de cola / buffer y estado.

Touch time <= Cycle Time <= Lead time

Eficiencia del Ciclo del Proceso (%) = Touch time /Elapsed Time (lead time)

Valores y principios

- **Valores** (se resumen en "respeto"): transparencia, equilibrio, colaboración, foco en el cliente, flujo, liderazgo, entendimiento, acuerdo, respeto.
- **Principios directores**: sostenibilidad, orientación al servicio, supervivencia.
- Idea de Covaro: las prácticas son lo más fácil de implementar, pero no aseguran que se respeten los principios. Sin los valores (cultura) es muy difícil que las prácticas permitan implementar los principios. Se vincula con *hacer ágil* vs *ser ágil*.

Scrum vs Kanban

Aspecto	Scrum	Kanban
Tiempo	Iteraciones de tiempo fijo	Tiempo fijo opcional; cadencia variable
Compromiso	El equipo se compromete a trabajo por iteración	Opcional
Métrica por defecto	Velocidad	Lead time

Aspecto	Scrum	Kanban
Roles	PO, SM y equipo (prescritos)	No prescribe roles
Equipos	Multifuncionales	Multifuncionales o especializados
Límite de WIP	Indirecto, marcado por el sprint	Directo por columna
Agregar trabajo	No se puede agregar alcance en el sprint	Se puede cuando hay capacidad
Estimaciones	Obligatorias	Opcionales
Tablero	Se limpia en cada sprint; cada equipo el suyo	Persistente; puede compartirse
Priorización	Product backlog priorizado	Opcional
Tamaño de funcionalidades	Deben entrar en un sprint	No prescribe
Gráficos	Burndown	No prescribe

Pregunta de parcial: principios Lean y prácticas de Kanban

Respuesta modelo del resumen final (revisar con la clase grabada, porque ese resumen tiene errores):

Principio Lean	Práctica(s) de Kanban que lo aplica
Eliminar desperdicios	Visualizar el trabajo (expone actividades sin valor y esperas que traban el flujo). Limitar el WIP (evita el trabajo a medias y el multitasking)
Amplificar el aprendizaje	Hacer explícitas las políticas (todos acceden a las reglas y al conocimiento). Mejora y evolución continua
Embeber la integridad conceptual	Sin respuesta en el resumen. Se puede apoyar en las políticas de calidad/DoD por columna *[Completar]*
Diferir compromisos hasta el último momento responsable	Limitar el WIP (en especial el de la columna backlog)
Ver el todo	Visualizar el trabajo (el tablero muestra todo el flujo y su sentido para el cliente)
Dar poder al equipo	Políticas explícitas + sistema pull: cada miembro toma el trabajo cuando está en condiciones, nadie se lo impone

Principio Lean	Práctica(s) de Kanban que lo aplica
Entregar lo antes posible	Gestionar el flujo (ver el recorrido completo y ajustar para entregar más valor en menos tiempo)

Unidad 2 – Métricas en los 3 enfoques

Una métrica es un número (valor cuantitativa) que representa el grado o presencia de atributos de lo que se quiere medir (proceso, producto o proyecto), de modo que permita una medición objetiva. Sirven para señalar y controlar el desarrollo, predecir al estimar, evaluar costos y mejorar el proceso, el proyecto o el producto.

Ideas que conviene tener claras:

- **Las métricas nunca se usan para evaluar o castigar a las personas.** Si se usan así, la gente las falsea.
- Toda métrica tiene un costo (planificarla, recolectarla, analizarla). Hay que hacer análisis costo-beneficio y no medir de más. Si no se usa, no vale el costo.
- Hay que preguntarse: ¿da más información que la que ya tengo?, ¿es de beneficio práctico?, ¿me dice lo que quiero saber?
- Elegir pocas y relevantes. "Si estás a miles de kilómetros del destino, no tiene sentido medir en milímetros".
- En software muchas características no se miden directamente (ej.: usabilidad), por lo que se miden fenómenos asociados.
- Se mide para saber dónde se está parado y si se mejora (analogía de la dieta: si no te pesás, no sabés si mejorás).
- Hay que tener el foco en mantenerlo simple y en que las métricas que vamos a definir sean aquellas que las personas necesitan y realmente van a utilizar. El consejo más interesante es, independientemente del enfoque, es que se pueda automatizar lo más posible la recolección de las métricas y que estas métricas tengan una línea clara de para que se van a usar.

Enfoque tradicional (procesos definidos)

Énfasis en los tres **dominios** de métricas. Hay un abanico amplio y el líder de proyecto decide cuáles tomar y cómo trabajarlas. Se definen en la planificación del proyecto.

Dominio	Foco	Quién / visibilidad	Ejemplos
Producto	Lo que se construye. Casi todas las de defectos	Equipo de desarrollo y testing. Propias de ese producto	Líneas de código, defectos por severidad, cantidad de casos de uso, promedio de métodos por clase
Proyecto (tácticas)	Cómo va la ejecución del proyecto; su utilidad termina	Líder de proyecto y equipo. Privadas del proyecto	Costos estimados vs reales, fechas reales vs programadas,

Dominio	Foco	Quién / visibilidad	Ejemplos
	cuando termina el proyecto		esfuerzo por tarea, eficiencia de estimación
Proceso (organizacionales o estratégicas)	Mejorar el proceso de la organización a largo plazo	Ingeniero de procesos. Públicas y despersonalizadas (promedio de muchos proyectos)	Desviación organizacional de estimaciones, defectos por severidad en la organización, propagación de errores de fase a fase

Las métricas de proyecto se consolidan y despersonalizan para convertirse en métricas de proceso: son los mismos datos vistos distintos. La diferencia es hacia qué apuntan. Esto solo tiene sentido en procesos definidos, porque asumen que la experiencia es extrapolable.

Métricas básicas de un proyecto de software:

Tamaño del producto (producto): Responde al que

funcionalidades en PUD y RUP en métricas de caso de uso por complejidad (tiene que ser un valor homogéneo),

esfuerzo (proyecto) Responde al cómo

Bolsa de horas persona lineales: una persona que hace el trabajo haciendo una sola cosa a la vez,

calendario/tiempo (proyecto): Responde al cuándo

La derivación del esfuerzo en el calendario no es directa (una mujer hace un bebe en nueve meses pero 9 mujeres no hacen u bebe en un mes) tiene en cuenta el índice de solapamiento y otros aspectos y

defectos (producto):

Medir sobre el producto cuantas cosas se no son coincidentes con lo que se espera, teniendo en cuenta la severidad

Se suman costos y riesgos como métricas tradicionales.

Enfoque ágil (procesos empíricos)

La medición es una salida, no una actividad.

Tiene una vision minimalista sobre las metricas: se mide solo lo estrictamente necesario. Dos principios guían la elección: *el software funcionando es la principal medida de progreso* y *satisfacer al cliente con entregas tempranas y continuas*. Por eso se mide sobre todo **producto**, y las métricas sirven al equipo (la experiencia no es extrapolable a otros equipos).

- **Velocidad**: puntos de historia que el equipo terminó y que el PO aceptó en el sprint (producto). Se **calcula** (no se estima) al final. No cuenta el trabajo parcialmente terminado. Al avanzar los sprints debería estabilizarse (los primeros sprints suelen ser bajos o cero). Se mira el promedio,

no el valor de un sprint. Se grafica con barras por sprint. Es la métrica de producto y la más importante del enfoque.

- **Capacidad**: compromiso de trabajo que puede asumir el equipo en un sprint. Se **estima** al inicio (por eso se la llama velocidad estimada), en puntos de historia si el equipo es maduro o en horas ideales/lineales si recién empieza. Se usa en el sprint planning para decidir cuántas US pasan del product backlog al sprint backlog. Es una métrica de **proyecto.**
- **Running Tested Features (RTF)**: cantidad de características (US, casos de uso, requerimientos) terminadas, testeadas y funcionando. Si la curva es llana o decrece hay un problema. Límite: no considera tamaño ni complejidad de cada pieza (medición absoluta). Conviene complementarla con velocidad.
- Las métricas de defectos también se usan en todos los enfoques.
- Los gráficos de burn-down, burn-up y combinado muestran el trabajo restante y completado.

Enfoque Lean (Kanban)

El foco es el **proceso**: Kanban es un flujo continuo sin proyecto con principio y fin, así que se mide el comportamiento del proceso en función del tiempo que demora cada pieza.

Métrica	Qué mide	Vista
Lead time (elapsed time)	Desde que el cliente pide algo (entra al backlog) hasta que se lo entregan. "Ritmo de terminación". La más importante para el cliente	Cliente
Cycle time	Desde que el equipo empieza a trabajar la pieza hasta que se entrega. Es el lead time menos el tiempo de espera en el backlog	Interna (equipo)
Touch time	Tiempo en que efectivamente se trabajó la pieza (columnas de producción), sin contar esperas (columnas de acumulación)	Trabajo real
Eficiencia del ciclo de proceso	Touch time / lead time. Cuanto más cerca de 1, más eficiente (poca espera)	Proceso

MÉTRICAS ORIENTADAS A SERVICIO KANBAN

Expectativa de nivel de servicio que los clientes esperan

Capacidad del nivel de servicio al que el sistema puede entregar.

Acuerdo de nivel de servicio que es acordado con el cliente.

Umbral de la adecuación del servicio el nivel por debajo del cual este es inaceptable para el cliente.

Resumen

Enfoque	Foco principal	Métricas
Tradicional	Proyecto, proceso y producto	Esfuerzo, tiempo, costos, riesgos, tamaño, defectos
Ágil	Producto (software funcionando)	Velocidad, capacidad, running tested features
Lean / Kanban	Proceso	Lead time, cycle time, touch time, eficiencia del proceso

En gestión de productos, las métricas de producto se dividen en **tamaño** (líneas de código, requerimientos: casos de uso, features, clases, puntos de historia) y **defectos** (cobertura, defectos por severidad, densidad de defectos).

Unidad 3 – Prácticas continuas

La cátedra aclaró que entran las **prácticas continuas**, pero NO las estrategias de despliegue (Canary y Blue/Green). Esta sección sale del resumen final, porque el resumen de Covaro no la cubre. *[Completar]* con la clase grabada.

Estas disciplinas son la evolución de la gestión de configuración, es decir que necesita la base de gestión de configuración como primera capa, ninguna de estas cosas es posible. Es llevar la evolución de SCM a un nivel de automatización que nos permite ir implementando cada una de estas prácticas continuas.

Contexto: en ambientes ágiles la gestión de configuración cambia de enfoque: sirve al equipo para seguir y coordinar el desarrollo, no para controlar a los desarrolladores. Responde a los cambios en lugar de evitarlos, automatiza donde se puede, da retroalimentación continua sobre calidad, estabilidad e integridad, está embebida en las tareas del sprint y es responsabilidad de todo el equipo. De las actividades de SCM, solo la **auditoría de configuración** no la soporta la filosofía ágil pura (tampoco el comité formal de control de cambios).

La diferencia entre estas 3 prácticas es el nivel de automatización de las pruebas

Práctica	Qué es	Detalle clave
Primer nivel Integración contínua (CI)	Los desarrolladores integran su código a un repositorio compartido varias veces al día. Cada integración se verifica con pruebas automatizadas con el fin de detectar problemas tempranamente. Asegura que el SW pueda ser DESPLEGADO en cualquier momento, que compile el código.	Detecta problemas temprano. Asegura que el software pueda desplegarse en cualquier momento (compila y tiene calidad). Cada desarrollador hace pruebas unitarias, idealmente con TDD, y sube el código al repositorio de integración. Genera el build
Entrega continua (Continuous Delivery)	El software se construye de forma que puede liberarse a producción en cualquier momento	Suma la automatización de las pruebas de aceptación. La CI es prerrequisito. Hay una decisión humana: el Product Owner decide si pasa a producción. El software siempre está en estado "entregable" (buildea, compila, los tests pasan). Despliegue automatizado para que cada puesta sea corta y de bajo riesgo
Despliegue continuo (Continuous Deployment)	Poner el producto en el ambiente de producción del usuario final. Cuando el último paso también se hace de forma automática.	A diferencia de la entrega continua, NO hay intervención humana: se usan pipelines con pasos que se ejecutan en orden. Busca la mayor TRANSPARENCIA para el usuario que puso en producción esta nueva versión del producto. PELIGRO en fallas que pueden ser llevadas a producción, preferiblemente se deben hacer despliegues rápidos y cortos para poder solucionar todos los defectos.

Cómo se relacionan: CI → Entrega continua → Despliegue continuo. Cada una agrega automatización. La diferencia entre ellas es el nivel de automatización de las pruebas y si la decisión de ir a producción es humana o automática.

Lo que NO entra: estrategias de despliegue (Canary deployment: liberar por etapas a un grupo pequeño de usuarios; Blue/Green: pasar el tráfico de la versión azul a la verde). Quedan solo como referencia.

Unidad 4 – Conceptos generales de calidad

Qué es la calidad

Conjunto de aspectos y características de un producto o servicio que le permiten satisfacer necesidades **explícitas e implícitas** (las implícitas son las que no se dicen en ningún lado, como "que la interfaz sea

agradable"). Es **subjetiva**: depende de la persona, el contexto y el momento. Por eso lo primero es hacer explícito qué espera realmente el cliente. Todas las acciones de aseguramiento apuntan a que no haya desviación entre lo que se le entrega al cliente y lo que espera. La calidad no se negocia.

La calidad no es solo del producto: también del **proyecto** y del equipo. Señales de falta de calidad: atrasos en las entregas, costos excedidos, incumplimiento de compromisos, requerimientos poco claros, software que no hace lo que debería, trabajo fuera de hora, el fenómeno 90-90 ("90% hecho, 90% faltante"), no aplicar SCM. Un proyecto sin calidad difícilmente logre un producto con calidad.

Un software de calidad satisface: expectativas del cliente, del usuario, necesidades de la gerencia y del equipo de desarrollo y mantenimiento, y de otros interesados.

Integridad vs calidad: un producto tiene integridad cuando satisface las necesidades del usuario, permite la trazabilidad en su ciclo de vida, cumple performance y requerimientos no funcionales y cumple expectativas de costo. No son sinónimos, pero están relacionados: no puede haber calidad sin integridad.

Por qué importa trabajar con calidad

Ventajas

- Satisface al cliente (el motivo principal) y lo retiene; incrementa los beneficios.
- Es un aspecto competitivo para sobrevivir en el mercado (las empresas certifican calidad de proceso, no de producto).
- Mejor costo-efectividad: menos fallas y menos retrabajo. Prevenir es más barato que corregir.
- Mejora la percepción del cliente (menos atrasos, sobrecostos, incumplimientos).

Desventajas

- **Costos**: de capacitación, de actividades de aseguramiento (revisiones, auditorías, testing, métricas) y de esfuerzo organizacional.
- Las revisiones y auditorías "parecen" ralentizar el proceso, sobrecargan el inicio y generan resistencia (burocracia, "se meten con mi trabajo").
- Exige compromiso en todos los niveles y cambio cultural; si falta, la transición es difícil.
- En la práctica, las empresas que compiten a veces prometen plazos ajustados y terminan sacrificando calidad.

[Completar] con las desventajas que se hayan dicho en la clase grabada.

Principios de calidad / aseguramiento

1. La calidad no se inyecta ni se compra: se embebe desde el momento cero (requerimientos en adelante). El testing solo la controla.
2. Es responsabilidad de todos los involucrados.
3. Las personas son la clave: capacitación ("si creés que capacitar es caro, probá con la ignorancia").
4. Se necesita un sponsor a nivel gerencial.
5. Se lidera con el ejemplo.
6. No se puede controlar lo que no se mide: usar métricas.
7. Simplicidad: empezar por lo básico.

8. Debe planificarse.
9. Aumentar las pruebas NO aumenta la calidad automáticamente.
10. Debe ser razonable para el negocio.
11. Prevención mejor que corrección.

Visiones de calidad (calidad para quién)

Visión	Idea
Del usuario	Expectativas del usuario. La más difícil de establecer (está en su cabeza). En ágil se incluye mediante el rol de Product Owner
Del producto	Nivel de satisfacción de los requerimientos del producto
Del proceso (manufactura)	Si el proceso usado es correcto y aporta valor sin generar desperdicio
Basada en el valor	Equilibrio costo-beneficio: máximo valor para el cliente y ganancias para quien desarrolla
Transcendental	Utópica (ej.: cero defectos); motiva la mejora continua. Responde "¿para qué quiero construir software?"

La cátedra dijo que el tema de perspectivas "no entra" (es de Scrum). Las visiones de calidad se dejan como contexto.

Calidad programada, necesaria y lograda

- **Programada:** lo que se planificó alcanzar.
- **Necesaria:** el mínimo que el producto debe tener para satisfacer los requerimientos.
- **Lograda:** la calidad real del producto terminado.

Cuanto más coinciden las tres, menos desperdicio e insatisfacción.

Calidad del proceso vs calidad del producto

- Lo que interesa es la calidad del producto, pero se insiste en la del **proceso** porque es el **único factor controlable** (tecnología, cliente, personas y entorno no se pueden controlar).
- **Proceso definido:** si el proceso tiene calidad y la gente lo respeta en el proyecto, el producto tendrá calidad. Todos los modelos de calidad se basan en procesos definidos.
- **Proceso empírico:** la calidad del producto depende de las personas; si el equipo hace lo que tiene que hacer, el producto tendrá calidad. No están de acuerdo con las auditorías externas; usan revisiones técnicas e inspección y adaptación.
- Un proceso es una "mesa de tres patas": lineamientos escritos + personas capacitadas y motivadas + herramientas y equipamiento. Que el proceso sea bueno no garantiza que la gente lo cumpla.
- Calidad de producto: no se puede sistematizar ni certificar (cada producto es único). Se asegura con revisiones técnicas y auditorías; se controla con testing.

Calidad del software = más que testing

QA ≠ testing. El aseguramiento actúa mientras se construye (preventivo, "lo que no está controlado no está hecho"); el testing llega cuando el producto ya está construido.

Unidad 4 – Aseguramiento y gestión de calidad

Gestión de calidad vs aseguramiento de calidad

Gestión de calidad: actividades que se realizan en una organización o proyecto para dar la calidad que necesita. Es lo que hace el gerente de calidad: hace el plan, el seguimiento, asigna gente y recursos. Tiene tres niveles de interés:

- **Organización:** marco de proceso y estándares que conduzcan a mejor software.
- **Proceso:** aplicar procesos específicos de calidad y verificar que se sigan.
- **Proyecto:** establecer un plan de calidad (metas, procesos y estándares a usar).

Aseguramiento de calidad (QA): acciones directas para que el producto y el proyecto tengan la calidad deseada. Define estándares, procesos y procedimientos y asegura que se respeten. Es una medida **preventiva**: se inserta en cada actividad para detectar lo antes posible oportunidades de mejora, tanto del producto como del proceso. El objetivo final es generar una cultura de calidad donde sea responsabilidad de todos.

Dentro de la gestión de calidad están tres actividades:

Actividad	Qué hace
Aseguramiento de calidad	Establece estándares y procedimientos de calidad. Elige contra qué estándares se compara. Las revisiones técnicas y las auditorías son actividades de comparación
Planificación de calidad	Selecciona los procedimientos y estándares para el proyecto y los modifica si hace falta. Define cuándo se harán revisiones y auditorías. Resultado: plan de calidad (uno de los planes de soporte del plan de proyecto)
Control de calidad	Ejecuta lo planificado y revisa en qué situación está el proyecto. Se hace con revisiones de calidad y con evaluaciones y mediciones automáticas. El testing es control de calidad

Estructura de un plan de calidad (Humphrey): introducción del proyecto, planes del producto (fechas de entrega y responsabilidades), descripciones de procesos, metas de calidad, riesgos y su gestión.

Objetivos de QA

- Realizar los controles apropiados del software y de su proceso de desarrollo.
- Asegurar el cumplimiento de estándares y procedimientos.
- Asegurar que los defectos en producto, proceso o estándares se informen a la gerencia para que los resuelva.

Funciones de QA

Evaluar la planificación del proyecto, los requerimientos, el proceso de diseño, las prácticas de programación, el proceso de integración y prueba, y los procesos de planificación y control de proyectos; desarrollar herramientas y técnicas de QA; y adaptar los procedimientos de QA a cada proyecto.

Estándares

Son importantes porque capturan la mejor práctica de la organización, son la base para decidir si se alcanzó el nivel de calidad y hacen que todos adopten las mismas prácticas. Hay dos clases:

- **De producto:** características que deben tener los componentes (estándares de documentos, de codificación).
- **De proceso:** procesos que deben seguirse en el desarrollo (requerimientos, diseño, validación, herramientas de soporte).

Proceso y proyecto: dónde se hace QA

El proceso se materializa en proyectos. Las actividades de aseguramiento (revisiones técnicas, auditorías) y de control (testing) se hacen **en el contexto de un proyecto concreto**. La disciplina de gestión de configuración da la base para poder trabajar con calidad. No se habla de aseguramiento de calidad *del proyecto* porque está implícito: el proyecto implementa un proceso.

Grupo de aseguramiento de calidad (GAC): independencia

- No reporta al líder de proyecto ni al gerente al que reportan los proyectos: pierde independencia y objetividad.
- Debe tener un reporte independiente, lo más cerca posible del nivel más alto de la organización: no más de un nivel de gerencia entre la dirección y el GAC.
- Debe reportar a alguien realmente interesado en la calidad.

Modelos para evaluar vs para mejorar vs para crear procesos

Tipo	Para qué	Ejemplos
Modelos de calidad (para crear procesos)	Referencia con lineamientos que el proceso debe seguir para llegar a un objetivo. No dicen cómo, dicen qué	CMMI, ISO 9001, SPICE
Modelos de mejora	Guían un proyecto cuyo resultado es un proceso mejorado	IDEAL, SPICE
Modelos de evaluación	Miden el grado de adherencia del proceso al modelo de referencia (auditoría o certificación)	SCAMPI (para CMMI)

Unidad 4 – Modelos de calidad y de mejora de procesos

Los modelos **no son prescriptivos**: dicen *qué* hay que lograr, no *cómo*. Se toman como referencia para definir el proceso de la organización. Aplican a **calidad de proceso**; los modelos de calidad de producto (ISO 25010, Barbacci, McCall) no entran al parcial.

Cómo se relacionan (flujo)

1. Una organización quiere definir o mejorar su proceso de desarrollo.
2. Arma un **proyecto de mejora de proceso**, guiado por un modelo de mejora (**IDEAL** o SPICE). Su resultado no es un producto sino un **proceso**.
3. Para decidir qué debe tener ese proceso, usa un **modelo de calidad** como referencia (**CMMI**, ISO 9001). El análisis de brecha (dónde estoy vs dónde quiero ir) es donde entran los modelos de calidad.
4. El proceso debe estar **escrito, definido, conocido por toda la organización y aprobado**, con guías de adaptación.
5. Ese proceso se **adapta** a cada **proyecto** (queda plasmado en el plan de proyecto). El proceso es la teoría; el proyecto es la práctica.
6. Evaluadores externos acreditados evalúan con un **modelo de evaluación** (SCAMPI para CMMI) si el proceso y su uso en proyectos son compatibles con el nivel elegido.
7. El proyecto produce el **producto**.

IDEAL (modelo de mejora)

Modelo cíclico para armar un proyecto de mejora de proceso. Lo creó el mismo grupo que CMMI. Las mejoras de proceso nunca son prioridad, por eso hace falta sponsor.

Fase	Qué se hace
Initiating (inicio)	Reconocer la necesidad de cambio, definir metas y buscar un **sponsor** (alguien con poder de decisión que además financie)
Diagnosing (diagnóstico)	Determinar el punto de partida. **Análisis de brecha** (GAP): dónde está la organización y adónde quiere llegar. Madurez actual y riesgos
Establishing (establecimiento)	Plan de mejora detallado: objetivos, prioridades, actividades, entregables, recursos y responsables
Acting (actuar)	Ejecutar el plan de acción, probando el proceso en un **proyecto piloto**
Learning (aprender)	Documentar resultados y lecciones. Si fue exitoso, extrapolar a toda la organización; si no, corregir. Reinicia el ciclo

CMMI (Capability Maturity Model Integration)

- Modelo de calidad para **mejora y evaluación** de procesos de desarrollo, mantenimiento y operación. Es **descriptivo**. Surge de buenas prácticas recopiladas de organizaciones exitosas. Es evolución de CMM (que solo tenía representación por etapas).
- **Acredita** un nivel de madurez o de capacidad (ISO 9001 *certifica*).

- Se basa en **procesos definidos**: evalúa la forma de trabajo de la organización, escrita en sus procesos.
- **Organización** = el área de la empresa que se quiere evaluar, no necesariamente toda la empresa.
- **Tres constelaciones:** Desarrollo (CMMI-DEV, la que se ve en la materia, Medir, monitorear y administrar procesos de desarrollo), Adquisición (ACQ: Guiar la adquisición de productos y servicios; usado cuando la empresa DELEGA) y Servicios (SVC, Guiar entrega de servicios internos y externos)).
- **22 áreas de proceso** (iguales en ambas representaciones). El nivel 1 no tiene áreas.
- Cada área de proceso tiene un objetivo genérico y de 1 a 4 específicos, con prácticas asociadas. Para acreditar un nivel hay que cumplir **todos** los objetivos de **todas** las áreas de ese nivel y los inferiores (todo o nada).

Dos representaciones

	Por etapas	Continua
Mide	**Madurez** de la organización como un todo	**Capacidad** de áreas de proceso individuales
Niveles	5 (1 a 5)	6 (0 a 5). Nivel 0 = el proceso no se ejecuta
Divide	Organizaciones inmaduras (nivel 1) y maduras (niveles 2 a 5)	No clasifica a la organización: cada área tiene su nivel
Ventaja	Una clasificación única facilita comparar organizaciones	Permite especializarse y mejorar las áreas que se elijan
Desventaja	Hay que hacer actividades en áreas que quizá no sean relevantes	Hay áreas que pueden quedar sin calidad; la organización puede seguir siendo "inmadura"
Origen	Continuidad del CMM original	El "gancho" para atraer a la gente de SPICE

Si tenés un proceso en nivel de capacidad alto pero otro del que depende está en 0 no tiene sentido (ej.: monitoreo en nivel alto con planificación en 0).

Niveles de madurez (rep por etapas)

Nivel	Nombre	Característica
1	Inicial	Procesos impredecibles, reactivos ("apagar incendios"). No hay visibilidad de cuándo termina, cuánto cuesta ni qué calidad tendrá. Éxito por esfuerzo personal. Organización **inmadura**
2	Administrado (gestionado)	Los proyectos tienen gestionados los requerimientos y los procesos se planifican, miden y controlan. Procesos caracterizados por proyecto

Nivel	Nombre	Característica
3	Definido	Procesos estandarizados, documentados y proactivos, caracterizados por la organización. Revisión de pares
4	Cuantitativamente administrado	Métricas de calidad y productividad; análisis estadístico del rendimiento de los procesos
5	Optimizado	Foco en mejora continua y optimización a través de los aumentos y mejoras tecnológicas innovadoras

A mayor madurez, mayor capacidad de cumplir objetivos y menor riesgo.

Foco de la materia: nivel 2. Si una organización alcanza el nivel 2 tiene madurez para administrar sus proyectos y el resultado será un producto previsible. Son disciplinas de gestión y de soporte:

Área de proceso (nivel 2)	Disciplina
REQM: administración de requerimientos	Gestión. Identificar unívocamente, actualizar, trazar y gestionar los cambios (no habla de especificación, esa es de nivel 3)
PP: planificación de proyecto	Gestión
PMC: monitoreo y control de proyecto	Gestión
MA: medición y análisis	Métricas
PPQA: aseguramiento de calidad de proceso y producto	Soporte
SCM: gestión de configuración	Soporte
SAM: administración de acuerdos con proveedores	Opcional: solo si se terceriza parte del desarrollo: **Tipo A** cumplen el mismo nivel de calidad. **Tipo B** no, y hay que proveerles lo necesario.

Roles y grupos: CMMI pide que exista alguien responsable de cubrir cada rol o grupo; se adaptan al tamaño de la organización.

SPICE (ISO/IEC 15504)

Modelo con origen en ISO. Mejora y evaluación de procesos de software por **capacidad de procesos individuales**, en 6 niveles (0 a 5). A diferencia de CMMI **no trae un conjunto predefinido de áreas de proceso**: permite evaluar los procesos que la organización tiene. Es modelo de calidad y de mejora.

ISO

ISO 9001 es una norma de calidad genérica que fija pautas para definir el proceso y **certifica** repetibilidad: que se define un proceso y siempre se ejecuta igual. Una auditoría ISO debería informar todas las diferencias entre el proceso definido y lo que la norma exige. La cátedra dijo que no hay que estudiarla a fondo.

SCAMPI (método de evaluación de CMMI)

Método formal para que una empresa sepa si alcanzó un nivel o capacidad CMMI. Está sistematizado para evitar interpretaciones distintas entre evaluadores. Incluye una etapa previa a la visita: el evaluador revisa en teoría que el proceso sea compatible con el nivel elegido (es caro ir a la organización). Luego verifica que los proyectos cumplan lo que dijeron en su plan.

Unidad 4 – CMMI y ágil: procesos definidos vs empíricos

CMMI y ágil son compatibles. Se incorporan las prácticas ágiles al proceso definido de la organización, pero ese proceso debe estar escrito, comunicado, capacitado, aprobado y ser verificable. Cada enfoque tiene que ceder algo.

Aspecto	Procesos definidos (CMMI)	Procesos empíricos (ágil)
Proceso	Está definido de antemano por un grupo de proceso de la organización. Se puede adaptar al proyecto, pero dentro de esos lineamientos	Se define al iniciar el proyecto y lo define el equipo. Se prueba y se ajusta en las retrospectivas. Siempre hay proceso, pero con otra concepción
Quién controla el proceso	La organización. Un equipo solo puede sugerir mejoras	El equipo
De qué depende la calidad del producto	Del proceso: si tiene calidad y la gente lo respeta, el producto tendrá calidad	De las personas: si el equipo hace lo que debe, el producto tendrá calidad
Aseguramiento de calidad	Viene alguien externo (auditorías)	Se resuelve dentro del equipo (retrospectiva, revisiones técnicas)
Experiencia	Se puede extrapolar a otros proyectos y equipos; por eso se estandariza y se toman métricas organizacionales	No se puede extrapolar: sirve a ese equipo
Foco	Proceso (y también producto)	Producto: software funcionando. No verifican si se siguió un proceso acordado
Mejora de procesos	IDEAL, SPICE	Kanban

Qué cede cada uno para poder acreditar CMMI con prácticas ágiles

- **Ágil cede:** las auditorías externas (para acreditar hay que hacerlas; el proyecto define períodos de auditoría externa), cierta trazabilidad de requerimientos (US y demás) y evidencias.
- **CMMI cede:** no exige un registro que no pertenece a la práctica ágil. Por ejemplo, la guía de Scrum no pide evidencia de la daily; se puede negociar con el evaluador, antes de su visita, que la daily cuenta como práctica de seguimiento del proyecto sin necesidad de minuta. El modelo dice *qué* hacer, no *cómo*, siempre que sea razonable.
- Hacer auditorías no vuelve a una organización menos ágil en lo cotidiano.

Incompatibilidades a medida que se sube de nivel: CMMI pide definir un proceso y cumplirlo (se verifica con auditorías); ágil nunca evalúa si seguiste el proceso definido. Desde nivel 3 o 4 CMMI pide métricas y estadísticas de proyectos y productos para medir el proceso; ágil dice que la experiencia no es extrapolable.

Similitudes (del resumen final): ambos buscan organizaciones de alto desempeño, ambos planifican, ninguno es completo ni aplica a cualquier proyecto, y ambos tienen reglas cuyo incumplimiento trae repercusiones.

Ojo con estos errores de parcial: la cátedra marcó como error común definir el proceso definido como "las mismas entradas dan las mismas salidas" (el resumen final lo dice así), y también decir que el enfoque tradicional no acepta cambios de requerimientos. La diferencia correcta es quién define y controla el proceso (organización vs equipo) y de qué depende la calidad.

Unidad 4 – Auditorías de software

Evaluación **objetiva e independiente** (hecha por alguien ajeno al equipo auditado) de productos o procesos para asegurar el cumplimiento de estándares, lineamientos, especificaciones y procedimientos, con un criterio objetivo que incluye documentación. Es una actividad de la disciplina de aseguramiento de calidad. Implican esfuerzo y costo, pero sus beneficios son mayores: opiniones independientes, detectar áreas de insatisfacción del cliente, dar visibilidad de los procesos a la gerencia, identificar fortalezas y oportunidades de mejora. Los empíricos están en contra (creen que el equipo se autocorrige, ej.: en la retrospectiva).

Tipos de auditoría

Mientras se desarrolla el producto se hacen tres:

Tipo	Sobre qué	Qué hace
Auditoría de configuración física	**Producto** (una línea base)	**Verifica**. Controla que el ítem de configuración, tal como está construido, cumpla con la documentación técnica que lo describe y con el plan de gestión de configuración: que el repositorio esté donde se dijo, que se respete el esquema de nombrado, que la documentación sea consistente con el código. Se hace **primero**. Si no sale bien, no se hace la funcional

Tipo	Sobre qué	Qué hace
Auditoría de configuración funcional	**Producto** (una línea base)	**Valida**. Audita la trazabilidad de requerimientos: que la funcionalidad y performance de cada ítem sean consistentes con la ERS, y que el código implemente **solo y todos** los requerimientos. Antes de empezar pregunta si está el reporte de la física. QA debe validar que la matriz de trazabilidad esté actualizada
Auditoría de proyecto	**Proceso**	Verifica que el proyecto se ejecutó con el proceso que se comprometió a usar (el definido en el plan, adaptado de la definición teórica de la organización). Se hace según el PACS (plan de aseguramiento de calidad). Contrasta evidencia con lo documentado (ERS, arquitectura, diseño)

En la práctica la física y la funcional apuntan al producto y la de proyecto al proceso. En Scrum el propio equipo puede hacer este chequeo en la retrospectiva; en enfoques tradicionales el auditor debe ser externo.

Auditoría al grupo de calidad (externa de QA): ¿quién controla al que controla? Una persona externa a la organización audita al área de calidad para ver si cumple los compromisos de auditorías y revisiones que dijo que haría a los proyectos (se acuerda cada 6 meses o una vez por año). Pide el plan de calidad organizacional y evidencia de que se hicieron las auditorías, se tomaron métricas, hay gestión de configuración, etc. Termina con la entrega del informe; el seguimiento del plan de acción lo hace luego el gerente de calidad.

Independencia

- **Objetivo:** el auditor no tiene condicionamientos respecto de lo auditado.
- **Independiente:** no depende jerárquica, funcional ni salarialmente del equipo o elemento auditado.
- Si los auditores son internos, deben depender de una unidad distinta de la que depende el líder de proyectos, con reporte directo a la gerencia de primer nivel.

Roles

Rol	Responsabilidades
Auditado (suele ser el líder de proyecto)	Propone/acuerda la fecha, participa, entrega evidencia, contesta dudas y el reporte, propone y comunica el plan de acción para las desviaciones
Auditor (de fuera del proyecto; puede ser el grupo de QA)	Acuerda la fecha, comunica el alcance, recolecta y analiza evidencia objetiva relevante y suficiente, realiza la auditoría, prepara el reporte y hace seguimiento de los planes de acción

Rol	Responsabilidades
Gerente de SQA	Prepara el plan de auditoría, calcula costos, asigna recursos, resuelve las no conformidades entre auditor y auditado. Gestiona al equipo de auditores

Etapas del proceso de auditoría

1. **Preparación y planificación:** no son sorpresa. El auditado la solicita (suele ser el líder de proyecto) y junto con el auditor definen fecha, alcance, objetivo y participantes. Se definen métricas de auditoría. Se prepara el checklist.
2. **Ejecución:** reunión de apertura (cómo será el proceso, lugar y hora). El auditor escucha lo que la gente dice que hace (evidencia subjetiva), mira la documentación y pide evidencia (objetiva), responde las preguntas y completa el checklist.
3. **Análisis y reporte de resultados:** el auditor arma el reporte; se evalúan resultados, se hace reunión de cierre y se entrega el reporte final. El auditado puede no estar de acuerdo con algunos puntos y se deja asentado.
4. **Seguimiento:** el auditado analiza las desviaciones y prepara un **plan de acción** que el auditor revisa y aprueba. Según el acuerdo, el auditor da seguimiento hasta que se cierren.

Herramientas y técnicas

- **Checklist:** preguntas tipo cerradas que garantizan que el foco sea el mismo independientemente de quién audite. Es el **mínimo** a controlar; el auditor puede preguntar más. Se arma desde el proceso. Incluye fecha, auditados con su rol, auditor, proyecto, fase actual, objetivo y alcance, preguntas. También se usan en revisiones técnicas.
- Muestreo (muestra representativa), revisión de registros, herramientas automatizadas.

Hallazgos (resultados de una auditoría)

Hallazgo	Qué es	Plan de acción
Buenas prácticas	Algo superador a lo esperado que el auditor destaca. Los informes suelen empezar por ahí	No
Desviación (no conformidad)	Algo que no se hizo, o no se hizo como el proceso lo definía. Dos grados: **no adecuado** (está mal) y **necesita mejora** (se hace, pero no del todo bien: ej.: hay plan pero está desactualizado) → Requiere plan de accion.	**Sí**, lo propone el auditado
Observación	Opinión del auditor sobre algo que cumple pero es riesgoso o podría terminar en desviación (ej.: técnica de estimación mala). Queda a criterio del equipo	No

Diferencia observación vs desviación necesita mejora: la observación es una opinión del auditor sobre algo que hoy sí cumple; la desviación necesita mejorar, es algo que el proceso exige y se hace de forma deficiente.

El **reporte de auditoría** informa cualquier desviación que genere disconformidad de un producto respecto de sus requerimientos y cualquier desviación al proceso definido o requerimientos documentados.

Métricas de auditoría

Esfuerzo por auditoría, cantidad de desviaciones, duración de la auditoría, cantidad de desviaciones clasificadas por área de proceso de CMMI.

Auditorías y ágil

Si se quiere acreditar CMMI hay que tener auditorías (ágil es una filosofía y no se "audita"). El proyecto define los períodos en que se harán auditorías externas. La auditoría de configuración es la única actividad de SCM que la filosofía ágil pura no soporta.

Unidad 4 – Revisiones técnicas (peer review)

Actividad realizada **por un par** (colega con formación técnica similar a la del autor; no es un jefe, ni el cliente, ni un auditor ni alguien de calidad) cuyo propósito es mejorar la calidad mediante la **detección temprana de errores** en cualquier artefacto: código, requerimientos, diseño, arquitectura, riesgos, estimaciones, planes. Es un proceso **estático** de verificación y validación (no ejecuta el software) y **no corrige** errores: los detecta. Busca evitar el retrabajo y propagación de defectos.

- **Se revisa el producto, nunca a la persona.** Hay que construir una cultura de apoyo, no de búsqueda de culpables (hay mucha resistencia a que alguien revise el trabajo propio).
- Se puede hacer sobre versiones incompletas y considerar otros atributos de calidad.
- Ágil también las usa (por ejemplo para reemplazar auditorías y no traer a alguien externo al equipo).
- Ejemplo: un programador junior pide a uno senior una revisión de un patrón de diseño.
- **Error vs defecto:** el error se encuentra en la misma etapa en que se cometió (lo busca la revisión). El defecto es un error que se trasladó a una etapa posterior (lo encuentra el testing). Siempre es mejor encontrar errores antes que defectos.

Ventajas: descubre muchos errores, permite inspeccionar versiones incompletas, reduce retrabajo y costos.

Desventajas: es difícil introducir las inspecciones formales, sobrecargan los costos al inicio y solo ahorran cuando los equipos tienen experiencia, requieren tiempo para organizarse y "parecen" ralentizar el desarrollo.

Formales vs informales

	Informales (walkthrough o recorrido)	Formales (inspecciones)
Proceso	No hay proceso definido ni control. Reuniones informales entre colegas	Proceso definido con roles
Roles	Sin roles	Autor, moderador, anotador, lector, inspector
Checklist	No	Obligatorio
Métricas	No obtiene métricas ni deja registros	Se toman métricas
Informe	No	Se entrega informe de la revisión
Objetivos	Mínima sobrecarga, capacitación de desarrolladores, rápido retorno	Descubrir errores, verificar que cumple requerimientos y estándares, software uniforme, proyectos más manejables
En ágil	Es de las más elegidas (revisión entre colegas al completar cada iteración)	Poco usadas

A mayor sistematicidad hay más probabilidad de encontrar defectos, pero algo informal igual es mejor que no hacer nada.

Inspección: etapas (entran en el parcial)

Roles y tiempos solo a nivel conceptual. Es un proceso time-boxed y de alto esfuerzo intelectual: las reuniones no deben superar las 2 horas.

1. **Planificación:** el moderador, a pedido del autor, define lugar, duración y roles.
2. **Visión general (opcional):** el autor describe el producto a inspeccionar.
3. **Preparación:** cada rol lee el producto y busca defectos potenciales antes de la reunión.
4. **Reunión de inspección:** el lector lee, los inspectores comparten defectos, el anotador los registra, se descartan falsos positivos y se concluye si se acepta o no el producto. Se hace un informe (qué se revisó, por quién, qué se encontró, conclusión).
5. **Corrección:** el autor corrige los defectos encontrados.
6. **Seguimiento:** si los defectos son graves, se re-inspecciona; si son simples, el autor corrige y a lo sumo se reúne con el moderador.

Roles (concepto): autor (creador del producto), moderador (planifica y lidera), anotador (registra hallazgos), lector (lee el producto), inspector (busca defectos antes de la reunión; todos pueden serlo). Un participante puede asumir varios roles.

Auditorías vs revisiones técnicas

Aspecto	Auditoría	Revisión técnica (inspección es un tipo de revisión técnica formal)
Para qué sirve	Revelar qué se hace en la realidad frente a lo que se comprometió hacer	Encontrar errores en un artefacto lo antes posible, para evitar su propagación
Quién	Auditor independiente, externo al proyecto (objetivo e independiente)	Un par; sin jerarquía
Qué compara	Lo hecho contra el proceso/estándar/línea base comprometidos	El artefacto contra lo que corresponda (ej.: arquitectura contra requerimientos)
Roles	Auditor, auditado y gerente de calidad	Autor, moderador, anotador, lector, inspector
Tipos	De proyecto, de calidad, de producto (física y funcional)	Formales e informales
Etapas	Planificación, ejecución, análisis y reporte, seguimiento	Informal: no hay pasos. Formal: planificación, visión general, preparación, reunión de inspección, corrección, seguimiento
Hallazgos	Buenas prácticas, observaciones y desviaciones	Errores
Beneficios	Opiniones independientes, visibilidad de procesos, detectar insatisfacción potencial del cliente	Detección de errores, evitar propagación, menos retrabajo, más calidad del artefacto
Equivalente en Scrum	Retrospectiva (proceso)	Review / revisiones entre colegas (producto)

Revisión técnica de código vs testing de caja blanca *(la cátedra pidió esta diferencia)*. Ambas miran el código, pero la revisión es **estática** (lo lee una persona, no se ejecuta; la hace un par; detecta errores tempranamente) y es aseguramiento de calidad. El testing de caja blanca **ejecuta** el programa con casos de prueba derivados de la estructura interna (caminos, sentencias, decisiones, condiciones) y detecta defectos; es control de calidad. *[Completar]* con cómo lo explique la clase grabada.

Unidad 4 – Testing de software

Qué es el testing

El testing es una disciplina de control de calidad centrada en la ejecución de pruebas sobre el producto. Este NO inyecta calidad al final del ciclo de vida ni puede garantizar ausencia de errores, sino que su objetivo es hacer visibles defectos presentes bajo la hipótesis de que efectivamente el software construido tiene defectos.

Proceso **destructivo** de ejecutar el software con la intención de **encontrar defectos**, cuya presencia se asume de antemano. El objetivo no es demostrar que el software funciona. Se empieza con una actitud negativa (hipótesis: el software es incorrecto). Un test es **exitoso cuando encuentra defectos**; por eso un desarrollo exitoso (con QA a lo largo de todo el proceso) lleva a un testing no exitoso. Es parte del control de calidad: se hace cuando el producto (o lo que se quiere testear) ya está construido. **QA ≠ testing**. El testing NO agrega calidad ni garantiza que no haya errores; solo la revela y da confianza. Cuesta entre el 30% y el 50% del costo de un software confiable. Puede empezar antes de codificar porque necesita los requerimientos para armar los casos de prueba.

Cabe resaltar que QA (Quality Assurance) NO es lo mismo que testing por mas que en el mercado lo usen de sinonimos. QA es el conjunto de actividades planificadas y sistematicas aplicadas a lo largo de TODO el proceso de construccion de software para orientar el desarrollo hacia la calidad del producto y proceso, pudiendo generar que el testing FRACASE ya que no se encuentran DEFECTOS (lo cual no sería algo malo).

Mitos: (1) "el testing demuestra que no hay errores", (2) "el propósito es demostrar que el programa hace lo que debe". Son falsos: un programa puede hacer lo que debe y aun así tener defectos, y hay error también cuando hace lo que no debería.

Los mitos del Testing son:

● El testing demuestra que el código no tiene errores → en realidad busca demostrar que si los tiene, revelando la mayor cantidad de errores posibles.

● El testing comienza luego de la codificación → El testing debe integrarse en etapas tempranas del ciclo de vida, ya que la detección tempra ahorra esfuerzo y recursos

● El testing asegura calidad → Falso, como se mencionó antes

● El tester es el enemigo del desarrollador

Verificación vs validación: verificación = ¿estamos construyendo el producto *correctamente*? (libre de defectos). Validación = ¿estamos construyendo el producto *correcto*? (el que el cliente pidió). Es un proceso de todo el ciclo de vida: arranca en revisiones de requerimientos y sigue con diseño, código y pruebas. Fallas existen por problemas de comunicación y limitaciones de memoria.

Principios del testing (Myers)

#	Principio	Por qué
1	Definir el resultado esperado es parte necesaria del caso de prueba	Sin resultado esperado, el "ojo ve lo que quiere ver" y un resultado erróneo se interpreta como correcto
2	Un programador debe evitar testear su propio programa (excepto pruebas unitarias)	No quiere encontrar sus defectos y puede repetir sus malentendidos del dominio
3	Una empresa de desarrollo no debe testear sus propios programas	Mismo motivo; además puede destinar menos recursos para cumplir calendario y costos
4	Inspeccionar minuciosamente los resultados de cada prueba	Cada caso puede revelar más de un defecto
5	Escribir casos para entradas inválidas e inesperadas, además de las válidas	Muchos errores aparecen con un uso nuevo o inesperado
6	Verificar también que el programa NO haga lo que no debe	Hay defectos secundarios no deseados
7	Evitar casos desechables (ad hoc)	Deben ser reproducibles y reutilizables para volver a testear tras las correcciones
8	No planear el esfuerzo asumiendo que no se encontrarán errores	La definición de testing presume su existencia
9	Donde ya hay más errores, es más probable que haya más	Pareto: ~80% de los defectos están en ~20% de los módulos; asignar más pruebas ahí
10	Las pruebas son creativas e intelectualmente desafiantes	Hay métodos, pero siempre hace falta creatividad

Otros principios de Covaro: el tester no es enemigo del programador (visibiliza defectos, no rompe el software); el testing exhaustivo es imposible; **paradoja del pesticida** (repetir siempre los mismos casos hace que pasen sin fallas pero los defectos reales queden ocultos); es dependiente del contexto; es una falacia decir que al terminar hay ausencia de errores; el testing no es una etapa final sino que debe empezar temprano.

¿Cuánto testing es suficiente?

El testing exhaustivo es imposible (tiempo y costo). Se decide según el **nivel de riesgo y costo**, y el **criterio de aceptación** acordado con el cliente (en ágil con el PO, en tradicional con el líder de proyecto): por costos, % de tests corridos sin fallas, ausencia de defectos de cierta severidad, pasar el conjunto de pruebas y la cobertura, *good enough* (cierta cantidad de fallas no críticas es aceptable), o defectos detectados parecidos a los estimados. Los riesgos definen qué se testea primero.

Defecto: severidad y prioridad

	Severidad	Prioridad
Qué es	Gravedad del defecto. Carácter técnico, la asigna quien testea	Urgencia/impacto en el negocio para resolverlo. Define el orden de atención según el cliente
Escala	Bloqueante: No permite la ejecución del SISTEMA COMPLETO. Crítico, Mayor (funciona pero no bien), Menor, Cosmética (formato, ortografía, visualización)	Urgencia, Alta, Media, Baja

Son **independientes**: depende del tipo de negocio. Ejemplo: un defecto cosmético puede ser prioridad baja en un sistema de inventarios pero alta en un sistema de marketing, donde la presentación es clave.

Casos de prueba y condiciones de prueba

- **Condición de prueba:** reacción esperada de un sistema ante un estímulo (entradas). Se enuncia **antes** que los casos; cada condición debe probarse con al menos un caso.
- **Caso de prueba:** secuencia de pasos para obtener un resultado esperado, con precondiciones, datos y resultado esperado. Es el artefacto más importante del testing. Se escribe una vez y sirve para infinitas pruebas, siempre con el mismo resultado esperado. Debe ser **reproducible**: si no se reproduce, no es un defecto. Se minimiza la cantidad de casos maximizando los defectos encontrados.
- Partes: objetivo (qué se controla), condiciones de prueba (datos de entrada y entorno) y resultado esperado.
- Se derivan de: documentos del cliente, información de relevamiento, requerimientos, especificaciones de programación y código. Cuanta más documentación, más fácil especificarlos. Se apunta a valores límite o esquina. Ninguna técnica es completa: hay que combinarlas.

Ciclos de prueba y regresión

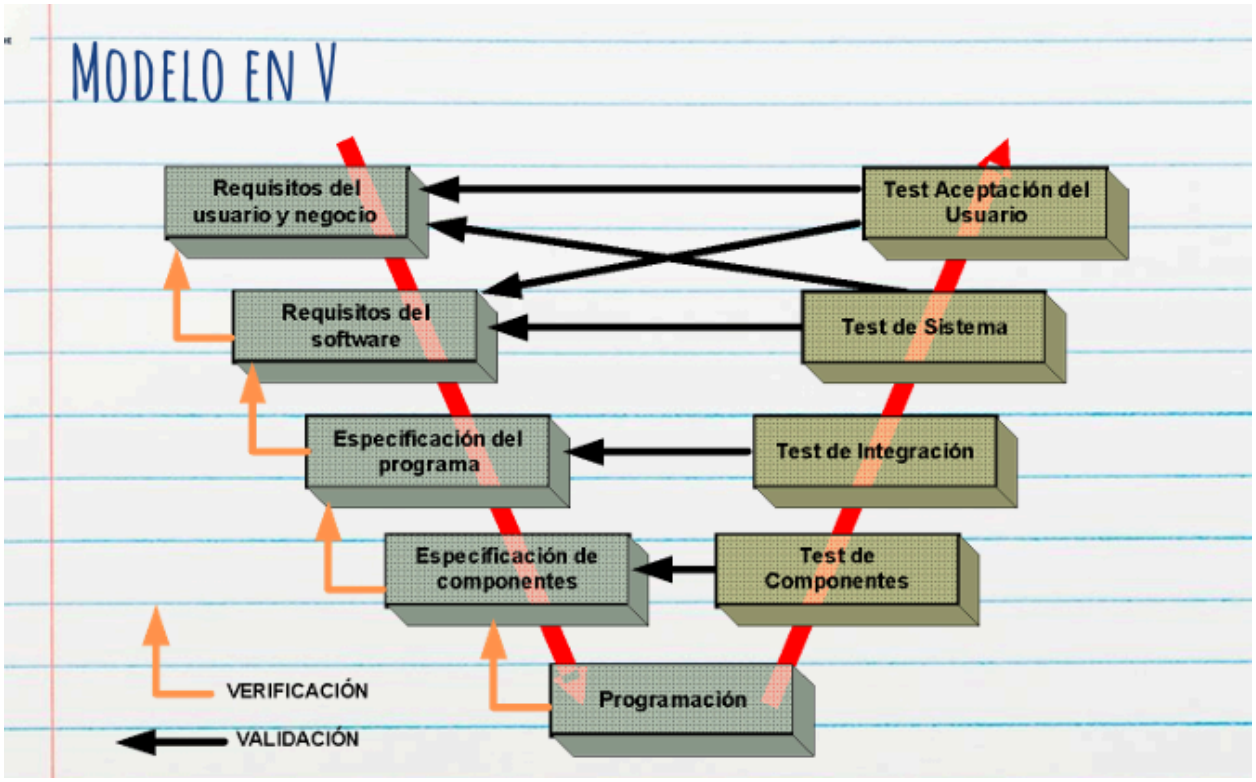
- **Ciclo de prueba/test:** ejecución del conjunto de casos de prueba sobre **una misma versión** del producto. Se corrigen los defectos, se hace otro ciclo con la nueva versión, y así hasta cumplir el criterio de aceptación. El **ciclo 0** es manual (se configura todo); desde el ciclo 1 se puede automatizar.
- Muchos ciclos indican que el QA no fue bueno (retrabajo y costos).
- **Regresión:** ejecutar de nuevo todos los casos en la nueva versión para prevenir los defectos nuevos que se introducen al corregir los detectados (la teoría dice que al corregir un defecto se introducen 2 o 3 más). Es lo ideal pero lo menos usado por tiempo y costo. Alternativa: ejecutar solo los casos con defectos previos y, cuando todo esté corregido, correr todos. La automatización permite hacer todos los ciclos con regresión que se quiera.

Niveles de prueba/testing

Se va subiendo de lo pequeño a lo grande (al revés de cómo se desarrolla).

Nivel	Qué prueba	Quién	Ambiente	Notas
Unitarias	Un componente individual y aislado	El desarrollador (único caso en que se acepta que pruebe su propio código). Incluidas en el DoD	Desarrollo	Con acceso al código. Fácil de automatizar. Se encuentran más errores que defectos
Integración	Dos o más componentes juntos, de forma incremental (top-down o bottom-up)	Tester	Pruebas (a veces desarrollo)	El resultado es el *build* (integración continua)
Sistema (o de versión)	El sistema completo: requerimientos funcionales **y no funcionales**	Personas ajenas al desarrollo	Preproducción (lo más parecido a producción). *Resumen final: ambiente de pruebas*	Casos de prueba; emula el entorno real
Aceptación de usuario	Que el sistema cumple lo que el usuario pidió	El usuario/cliente (en la práctica, las arma testing)	Preproducción o similar a producción	Objetivo: **generar confianza**, no encontrar fallas. Pruebas alfa (laboratorio) y beta (ambiente real). En ágil, con el equipo en la sprint review

Modelo en V: relaciona cada etapa de desarrollo con su nivel de prueba, en granularidad inversa (requerimientos ↔ aceptación, diseño/arquitectura ↔ integración y sistema, componentes ↔ unitarias).



Ambientes

Ambiente	Para qué	Pruebas
Desarrollo	Los desarrolladores crean, prueban y depuran. Flexible	Unitarias
Pruebas	Los testers hacen pruebas exhaustivas (los desarrolladores no deben tener acceso). Réplica parcial de producción	Integración, funcionales, performance
Preproducción (puesta en escena / calidad)	Prueba final con condiciones de producción: estrés, carga, seguridad. A veces es muy caro o imposible replicarlo	Sistema (Covaro) y aceptación
Producción	Usuarios finales usan el software. No se prueba acá	–

⚠️ Covaro asigna *sistema* a preproducción y *aceptación* a preproducción o similar a producción; el resumen final pone *sistema* en el ambiente de pruebas. Usar Covaro y confirmar en clase.

Proceso de pruebas/Proceso del testing (enfoque tradicional, proceso definido)

Testing ad hoc = sin proceso definido, sin registro ni trazabilidad de lo probado. El proceso definido tiene estas etapas:

1. **Planificación y control:** cómo entra el testing en el plan del proyecto. Resultado: **plan de pruebas** (riesgos y objetivos, estrategia, recursos, criterio de aceptación, entornos, quién hace qué y cuándo, herramientas, si hay regresión). Solo necesita los requerimientos, no el código.

Ppt:
La Planificación de las pruebas es la actividad de verificar que se entienden las metas y los objetivos del cliente, las partes interesadas (stakeholders), el proyecto, y los riesgos de las pruebas que se pretende abordar.

- Construcción del Test Plan:
    - Riesgos y Objetivos del Testing
    - Estrategia de Testing
    - Recursos
    - Criterio de Aceptación
- Controlar:
    - Revisar los resultados del testing
    - Test coverage y criterio de aceptación
    - Tomar decisiones

2. **Identificación y especificación (diseño) de casos de prueba:** se identifican datos, se diseñan y priorizan los casos, se analiza si los requerimientos son testeables y se define si se usa regresión.

Ppt:
- Revisión de la base de pruebas
- Verificación de las especificaciones para el software bajo pruebas
- Evaluar la testeabilidad de los requerimientos y el sistema
- Identificar los datos necesarios
- Diseño y priorización de los casos de las pruebas
- Diseño del entorno de prueba

3. **Ejecución:** manual y/o automatizada; se registran y comparan resultados reales vs esperados y se genera el reporte de defectos. Incluye crear datos, implementar y verificar el ambiente.

Ppt:
Desarrollar y dar prioridad a nuestros casos de prueba
- Crear los datos de prueba
- Automatizar lo que sea necesario
- Creación de conjuntos de pruebas de los casos de prueba para la ejecución de la prueba eficientemente.
- Implementar y verificar el ambiente.
- Ejecutar los casos de prueba
- Registrar el resultado de la ejecución de pruebas y registrar la identidad y las versiones del software en las herramientas de pruebas.
- Comparar los resultados reales con los resultados esperados.

4. **Análisis de fallas (evaluación y reporte):** seguimiento de la corrección hasta cerrar los casos. Se evalúa el criterio de aceptación y se reporta a los interesados. Si hay defectos se vuelve a ejecutar.

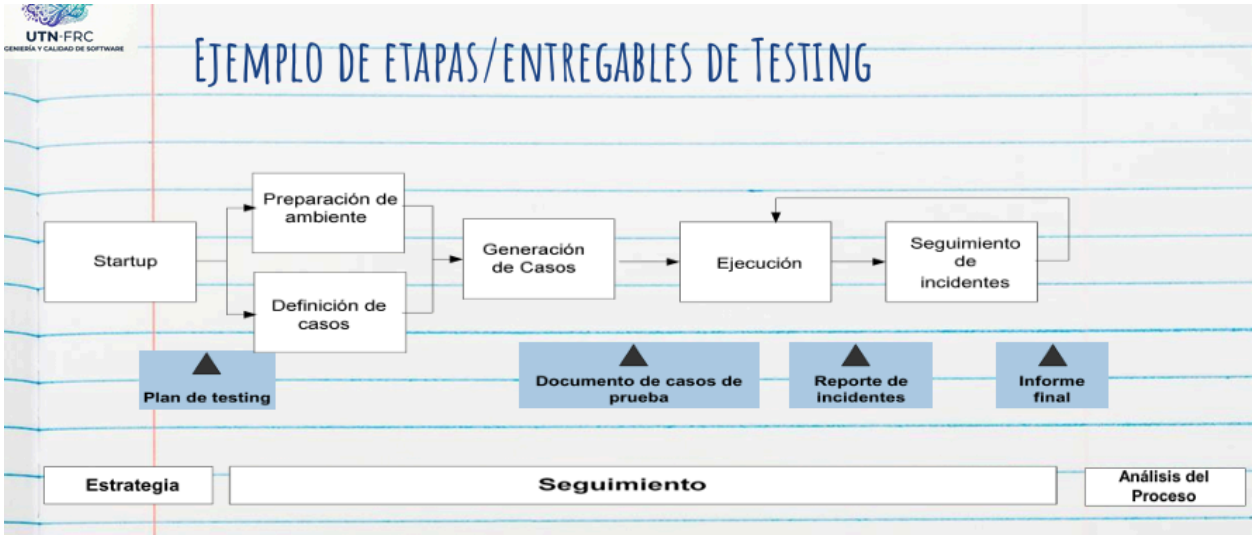
Ppt:

• Evaluar los criterios de Aceptación
• Reporte de los resultados de las pruebas para los stakeholders.
• Recolección de la información de las actividades de prueba completadas para consolidar.
• Verificación de los entregables y que los defectos hayan sido corregidos.
• Evaluación de cómo resultaron las actividades de testing y se analizan las lecciones aprendidas.

5. **Fin de las pruebas:** cuando se cumple el criterio de aceptación (en empresas maduras, sin defectos bloqueantes, críticos ni mayores; los menores y cosméticos van en la nota de release). Informe final (opcional).
6.

Artefactos: plan de pruebas/testing, documento de casos de prueba, reporte de incidentes o defectos (qué falló y cómo reproducirlo), informe final (métricas: ciclos, errores por ciclo, tipos de prueba; es el único negociable en algunas organizaciones).

En ciclo de vida secuencial las pruebas se hacen al final; en iterativo/incremental, por iteración (riesgo: errores por integrar incrementos).



Estrategias / métodos de prueba

Buscan cubrir la mayor cantidad de funcionalidad con la menor cantidad de casos (el tiempo y presupuesto son limitados). Son **complementarias**; conviene combinar técnicas.

	Caja negra	Caja blanca
Mira	Entradas y salidas; no la estructura interna	El código (o pseudocódigo/diagrama de flujo) y su estructura

	Caja negra	Caja blanca
Idea	Para ciertas entradas se espera ciertas salidas; si se obtienen, el caso pasa	Diseñar casos que recorran los caminos lógicos con la menor cantidad de casos
Debilidad	Testing exhaustivo de entradas es inviable	Hay muchísimos caminos lógicos; ni el recorrido exhaustivo asegura que el programa cumpla la especificación, ni detecta caminos faltantes
Técnicas	**Basadas en especificaciones:** partición de equivalencias, análisis de valores límite. **Basadas en experiencia:** adivinanza de defectos, testing exploratorio	Cobertura de enunciados/caminos básicos, de sentencias, de decisión, de condición, de decisión-condición, múltiple

Caja negra: partición de equivalencias

Se analizan las **condiciones externas** (entradas y salidas) y se dividen en **clases de equivalencia**: subconjuntos de valores que producen resultados equivalentes.

1. Identificar condiciones externas de entrada y salida (si hay dos datos del mismo tipo, se separan: calle1 y calle2).
2. Identificar subconjuntos válidos e inválidos. Los inválidos se asocian a mensajes de error/advertencia (la salida). La unión de todos los subconjuntos debe ser el universo de la condición externa.
3. Armar casos tomando un valor concreto de cada subconjunto.

- **Análisis de valores límite:** variante que usa los bordes de las clases de equivalencia.
- **Adivinanza de defectos:** por intuición y experiencia, se arma una lista de situaciones propensas a error.
- **Testing exploratorio:** el tester aprende el sistema mientras prueba y genera nuevas pruebas (no se puede automatizar).

Cómo se escribe un caso de prueba: nombre claro del escenario (ej.: "Ingresar al sitio con edad menor a 18"); prioridad (alta para caminos felices y errores críticos del negocio, baja para validaciones de formato); **precondiciones** con valores concretos ("El usuario Juan está logueado con permiso de administrador"); **pasos** sin ambigüedad ("El usuario ingresa a la opción…"), que arrancan por la opción que dispara la funcionalidad; **resultado esperado** concreto y específico ("El sistema muestra el mensaje de advertencia 'Debe seleccionar una fecha'"), relacionado con el paso donde se produce.

Caja blanca: coberturas

- **Caminos básicos (cobertura de enunciados):** se dibuja el **grafo de flujo** y se calcula la **complejidad ciclomática** M, que da la cantidad de caminos independientes y el mínimo de casos de prueba para ejecutar cada instrucción al menos una vez.

```
M = E - N + 2P
```

E = aristas, N = nodos, P = componentes conexos (nodos de salida). Alternativa: M = regiones cerradas + 1. Después se define el conjunto de caminos independientes, se diseñan los casos para cada uno y se ejecutan. No funciona con algoritmos recursivos. La cátedra mostró una clasificación del programa según el valor de M (*[Completar]* los rangos: en el resumen es una imagen).

- **Cobertura de sentencias:** mínimo de casos para ejecutar todas las **sentencias** (asignaciones, invocaciones). Las estructuras de control no son sentencias.
- **Cobertura de decisión:** casos para que cada **decisión** (el IF completo) evalúe verdadero y falso. No importa qué condición la compone. En un IF se necesitan al menos 2 casos.
- **Cobertura de condición:** cada **condición** (cada término lógico dentro de la decisión, ej.: A>0) evalúa verdadero y falso, sin importar la rama.
- **Cobertura de decisión-condición:** se cumplen ambas a la vez.
- **Cobertura múltiple:** todas las combinaciones de verdad de las condiciones. Con decisiones dependientes (anidadas) hay más casos (ejemplo del resumen: 7); con independientes, menos.

Tipos de prueba ojo aca, RECORDAR QUE EN CLASES HABIA UNA DUDA RESPECTO SI TESTING FUNCIONAL/NO FUNCIONAL ES UN TIPO O FOCO

Tipo o foco?	Qué es o tipo?
Smoke test	Primera corrida de las pruebas de sistema, rápida, antes del ciclo 0. Verifica que las funcionalidades básicas funcionen y que no haya fallas groseras. Si falla, no se empieza a testear formalmente
Sanity test	Se hace después de una ronda más exhaustiva, para verificar que se corrigieron los problemas críticos y que las correcciones no introdujeron problemas nuevos ni rompieron lo clave
Funcional (¿qué hace el sistema?)	Las pruebas se basan en funciones y características (descripta en los documentos o entendidas por los testers) y su interoperabilidad con sistemas específicos. **Basado en requerimientos** (prueba una funcionalidad; criterios de aceptación de una US) o **basado en procesos de negocio** (prueba el proceso completo: buscar, seleccionar, facturar) 1. Operacion normal 2. Pruebas negativas 3. Pruebas de proceso de negocio: Concatenar funcionalidades que permiten armar el circuito para pro… el proceso de negocio 4. Documentación: Manual de usuario sea consistente con todo el sistema (la última versión)

Tipo o foco?	Qué es o tipo?
	Creo que estos tipos de pruebas hacen foco en PROBAR req funcionales. No es que estos sean pruebas del TIPO FUNCIONAL.
No funcional (¿cómo lo hace?)	Hacen foco en requerimientos no funcionales. Depende del entorno, que debe ser lo más parecido al del cliente. Incluye: 1. Pruebas de seguridad: seg logica y fisica. Diseñar casos de prueba que están específicamente dirigidos a esto. Hacking ético son pruebas de vulnerabilidad 2. Pruebas de performance/desempeño (tiempos de respuesta y recursos) 3. Pruebas de escala completa → Pruebas de carga (BD):Usabilidad comprometida (comportamiento de hardware y comunicaciones), usuarios concurrentes, la “prueba de operación normal” debería pasar. 4. Stress (forzar al sistema más allá de lo normal; robustez y tiempo de recuperación). 5. Pruebas de usabilidad: Asumimos la funcionalidad, foco en el sentimiento del usuario (hay que tener en claro el perfil del usuario final). Prod min adorable hace foco en esto. → Prueba de accesibilidad: Orientada a cuantos usuarios pueden usar el sistema, no exclusivamente dirigido a atacar discapacidad, puede ser por ejemplo dirigido a adultos mayores. 6. Pruebas de mantenimiento: Ante un cambio en los requerimientos, que tanto esfuerzo requiere. Que tan mantenible es el sistema. 7. Pruebas de fiabilidad: Que tanto podes depender de los resultados que el sistema provee 8. Pruebas de portabilidad: Que tanto soporta a cambio de SO (acotado a los entornos comprometidos) 9. Pruebas de interfaz de usuario: Las GUIs son mucho mas complejas que las interfaces basadas en caracteres 10. Pruebas de configuración

Tipo o foco?	Qué es o tipo?
	Importante: Cada so tiene como críticos distintos de estos aspectos

TDD (Test Driven Development)

Práctica de desarrollo centrada en el desarrollo funcional. Dos prácticas: **Test First Development** (primero se escriben las pruebas unitarias de la característica; después el código para que pasen; obliga a modularizar y a que cada método/clase tenga una responsabilidad) y **Refactoring** (reestructurar el código sin cambiar su comportamiento, para mejorar claridad y reducir complejidad). Ciclo: **Red** (el test falla) → **Green** (el test pasa) → **Blue** (refactor). Se elige el requerimiento, se escribe la prueba, se implementa para que pase y luego se refactoriza. Si no tenés claro qué hay que hacer, no podés escribir la prueba.

Unidad 4 – Testing en ambientes ágiles

Se apoya en el **Manifiesto de Testing Ágil**, parecido al Manifiesto Ágil. Valora: testear a lo largo de todo el desarrollo sobre testear al final; **prevenir bugs** sobre encontrar bugs; entender lo que se testea (darle lógica) sobre solo verificar una funcionalidad; construir el mejor sistema sobre romperlo; la **responsabilidad del equipo** por la calidad sobre la responsabilidad del tester. No hay un rol fijo de tester: cualquiera puede tomarlo.

Principios del testing ágil (conceptuales)

1. El testing se mueve hacia adelante en el proyecto: se hace de forma constante al terminar cada iteración, no al final.
2. El testing no es una fase: se hace mientras se desarrolla el código.
3. Todos hacen testing.
4. Reducir la latencia del feedback.
5. Las pruebas representan expectativas.
6. Mantener el código limpio y corregir los defectos rápido (prevenir errores antes de solucionarlos).
7. Reducir la sobrecarga de documentación en las pruebas.
8. Las pruebas son parte del "Done" (el DoD incluye realizar las pruebas).
9. Pasar de probar al final a ser "conducido por pruebas".

Prácticas del testing ágil (concretas)

- **Testing unitario y de integración automatizado:** todo lo que se ejecuta de forma mecánica se automatiza (ahorra tiempo y recursos).
- **Testing regresivo a nivel de sistema automatizado.**
- **Testing exploratorio:** no se puede automatizar.
- **TDD** (Test Driven Development).
- **ATDD** (Acceptance Test Driven Development): desarrollo conducido por las pruebas de aceptación.

- **Control de versión de las pruebas con el código:** los unit tests y system tests viven junto al código, lo que facilita automatizar.

Cuadrantes del testing ágil

Mapa de los tipos de testing: cuál es el foco de cada uno, hacia dónde apuntan y cuáles se automatizan o hacen manualmente.

Cuadrante	Orientación	Tipos de prueba	Cómo
Q1	Tecnológico, soporta al equipo	Unit tests, component tests	Automatizado
Q2	Negocio, soporta al equipo	Pruebas funcionales, ejemplos, story tests, prototipos, simulaciones	Automatizado y manual
Q3	Negocio, critica el producto	Testing exploratorio, escenarios, usabilidad, UAT (pruebas de aceptación de usuario), alfa/beta	Manual
Q4	Tecnológico, critica el producto	Performance y carga, seguridad, atributos "-ility" (no funcionales)	Con herramientas

Los cuadrantes 1 y 4 son más tecnológicos; el 2 y el 3, de negocio. Los 1 y 2 soportan al equipo; los 3 y 4 critican lo que se hace sobre el producto.

Contexto en ágil (resumen)

- Se testea por iteración. Pruebas unitarias en el DoD. Pruebas de aceptación en la sprint review con el PO y el equipo.
- Los criterios de aceptación de las US y las pruebas de aceptación escritas antes de programar (recomendado que las escriba el cliente) son la base de los casos de prueba.
- La pirámide del testing (ágil vs tradicional), la automatización y los roles y competencias del tester se vieron en la clase de repaso/pecha kucha. *[Completar]* con la clase grabada, no están en los resúmenes.

Cuadros comparativos de repaso final

Calidad del producto vs calidad del proceso

	Calidad del producto	Calidad del proceso
Aseguramiento	Revisiones técnicas (por pares) y auditorías de configuración física y funcional	Revisiones técnicas y auditorías de proyecto
Control	Testing (el producto ya está hecho; muestra cuán lejos o cerca se está de los requerimientos)	–

	Calidad del producto	Calidad del proceso
Ceremonia de Scrum	Review	Retrospectiva
Contexto	Ambos se hacen en el contexto de un proyecto	
Modelos	No se puede sistematizar: ISO 25010, Barbacci/SEI, McCall son modelos teóricos (no entran)	CMMI, ISO 9001, SPICE. Lo que le sirve a una organización puede no servirle a otra
Certificación	No existe en la industria (cada producto es único)	Se acredita o certifica el grado de adherencia del proceso al modelo de referencia
Estándares	De producto: características de los componentes (codificación, documentación)	De proceso: cómo se implementan los procesos de desarrollo
Modelos de mejora	–	SPICE, IDEAL

Aseguramiento de calidad de proceso vs de producto

	De proceso	De producto
Foco	Evaluar y mejorar el proceso de desarrollo (cómo se hacen las actividades)	Evaluar y garantizar la calidad del producto resultante (artefactos y entregables)
Objetivo	Prácticas efectivas, eficientes y consistentes que optimicen productividad y minimicen errores	Que el producto cumpla requisitos, sea confiable, funcione y sea adecuado al uso previsto
Actividades clave	Estándares, buenas prácticas, revisiones técnicas y auditorías de proceso, métricas del proceso, gestión de configuración	Pruebas (funcionales, performance, seguridad), revisiones de código, auditorías de producto, validación con usuarios

Otras diferencias que suelen preguntarse

Concepto A	Concepto B
QA: preventivo, se hace durante todo el ciclo de vida, define procesos y estándares	**Testing:** control, llega cuando el producto está construido, busca defectos

Concepto A	Concepto B
Error: se detecta y corrige en la misma etapa	**Defecto:** error que se trasladó a una etapa posterior (lo encuentra el testing)
Verificación: ¿construimos el producto *correctamente*?	**Validación:** ¿construimos el producto *correcto*?
Severidad: gravedad técnica, la define el tester	**Prioridad:** urgencia para el negocio, la define el cliente
Auditoría física: verifica contra la documentación y el plan de SCM	**Auditoría funcional:** valida contra los requerimientos (trazabilidad)
Caja negra: entradas y salidas	**Caja blanca:** estructura interna del código
Smoke: antes del ciclo 0, funcionalidades básicas	**Sanity:** después de una ronda de pruebas, verifica correcciones críticas
Velocidad: se calcula al final, SP aceptados por el PO	**Capacidad:** se estima al inicio, compromiso del equipo para el sprint
Lead time: vista del cliente (backlog hasta entrega)	**Cycle time:** vista interna (desde que el equipo empieza)
Desviación: requiere plan de acción	**Observación:** opinión del auditor, no requiere plan de acción
Walkthrough: informal, sin roles ni métricas	**Inspección:** formal, con roles, checklist, métricas e informe
Continuous Delivery: decisión humana de ir a producción	**Continuous Deployment:** automático, sin intervención humana

Gestión tradicional vs gestión ágil (contexto de métricas y estimaciones)

Aspecto	Tradicional	Ágil
Respuesta a cambios	Poco flexible (acepta cambios, pero son costosos)	Altamente flexible
Incertidumbre	Baja: se elimina en requerimientos	Alta: se elimina cuando hace falta
Entrega de software útil	Tardía	Temprana
Retroalimentación del cliente	Baja	Alta
Quién estima	El líder de proyecto	El equipo de desarrollo

Aspecto	Tradicional	Ágil
Equipo	Jerárquico, con roles definidos	Autogestionado
Proceso	Definido	Empírico
Ciclos de vida	Cualquiera	Iterativo e incremental
Se estima	Recursos y tiempos para el alcance	Alcance (tamaño) a implementar en el tiempo fijo
Constantes	Los requisitos	Equipo, recursos y tiempo

Pendientes para completar con las clases grabadas

Buscá la marca *[Completar]* en el documento. Resumen de lo que más falta:

- ☐ Prácticas continuas (CI/CD): solo salen del resumen final; confirmar qué profundidad piden y si Despliegue continuo entra como práctica.
- ☐ Clase de servicio "intangible" de Kanban y respuesta para el principio Lean "embeber la integridad conceptual".
- ☐ Rangos de complejidad ciclomática (la tabla de clasificación es una imagen en el resumen).
- ☐ Pirámide del testing (ágil vs tradicional), automatización del testing y roles/competencias del tester (pecha kucha): no están en ningún resumen.
- ☐ Desventajas de trabajar con calidad: la clase de repaso solo dejó anotado "costos".
- ☐ Confirmar si las *visiones de calidad* entran (la nota dice que las perspectivas "no entran, es un tema de Scrum" y no queda claro si habla de este tema).
- ☐ Ejemplos propios de cada desperdicio Lean y de cada principio Lean.
- ☐ Confirmar qué se pregunta de la inspección (la cátedra dijo: etapas sí, roles y tiempos solo a nivel conceptual).

Diferencias entre los dos resúmenes (cuál usé)

Tema	Resumen de Covaro	Resumen final	Qué hice
Cycle time	Dice que "contiene" el tiempo en cola	Es lead time menos el tiempo en backlog	Usé la definición del resumen final (la de Covaro parece un error)
Velocidad	"Suma el esfuerzo estimado" de las historias	Suma los story points de las US aceptadas por el PO	Usé story points
Costo del testing	30% a 50% (y en otro lugar 20% a 50%)	30% a 50%	Usé 30% a 50%

Tema	Resumen de Covaro	Resumen final	Qué hice
Prácticas de Kanban	Seis: visualizar, limitar WIP, gestionar flujo, políticas explícitas, mejorar colaborativa y evolutivamente, respetar roles	Visualizar, limitar WIP, gestionar flujo, políticas explícitas, mejorar y evolucionar, circuitos de retroalimentación	Usé las de Covaro y mencioné la diferencia
Ambiente de las pruebas de sistema	Preproducción	Ambiente de pruebas	Usé Covaro
Definición de proceso definido	Se centra en que la organización lo define, controla y lo adapta a proyectos (según la clase de repaso)	"Mismas entradas, mismas salidas" (la cátedra lo marcó como error común)	Usé la versión de la clase de repaso
Representación continua de CMMI	Capacidad de áreas individuales, niveles 0 a 5	Igual	Sin diferencia

Recordá que el resumen final tiene otros errores que no detecté. Si algo de esta guía contradice la clase grabada, la clase gana.